

Taller GPU - Pantallazos de los resultados obtenidos

Stiven Aguirre Granada

Configuración del entorno de trabajo:

```
import torch
print(f"GPUs Available: {torch.cuda.device_count()}")

... GPUs Available: 1

!nvidia-smi

Sat May 30 02:34:31 2026
+-----+-----+-----+-----+-----+-----+
| NVIDIA-SMI 580.82.07              Driver Version: 580.82.07      CUDA Version: 13.0     |
+-----+-----+-----+-----+-----+-----+
| GPU   Name                Persistence-M   Bus-Id        Disp.A   Volatile Uncorr. ECC |
| Fan  Temp  Perf            Pwr:Usage/Cap     Memory-Usage | GPU-Util  Compute M. |
|=====+=====+=====+=====+=====+=====+
|    0   Tesla T4              Off          00000000:00:04:0  Off      0%          Default |
| N/A   43C    P8              9W /   70W           0MiB / 15360MiB |              MIG M. |
|=====+=====+=====+=====+=====+=====+
| Processes:                                                                  |
|  GPU   GI    CI          PID    Type   Process name                      GPU Memory |
|  ID   ID   ID                                            Usage      |
|=====+=====+=====+=====+=====+=====+
| No running processes found
+-----+-----+-----+-----+-----+-----+

%pip install dask-cuda

... Requirement already satisfied: dask-cuda in /usr/local/lib/python3.12/dist-packages (26.2.0)
Requirement already satisfied: click>=8.1 in /usr/local/lib/python3.12/dist-packages (from dask-cuda) (8.4.0)
Requirement already satisfied: cuda-core==0.3.* in /usr/local/lib/python3.12/dist-packages (from dask-cuda) (0.3.2)
Requirement already satisfied: numba-cuda<0.23.0,>=0.22.1 in /usr/local/lib/python3.12/dist-packages (from dask-cuda) (0.22.2)
Requirement already satisfied: numpy<3.0,>=1.23 in /usr/local/lib/python3.12/dist-packages (from dask-cuda) (2.0.2)
Requirement already satisfied: nvidia-ml-py==12 in /usr/local/lib/python3.12/dist-packages (from dask-cuda) (13.595.45)
Requirement already satisfied: pandas>=1.3 in /usr/local/lib/python3.12/dist-packages (from dask-cuda) (2.2.2)
Requirement already satisfied: rapids-dask-dependency==26.2.* in /usr/local/lib/python3.12/dist-packages (from dask-cuda) (26.2.0)
Requirement already satisfied: zict>=2.0.0 in /usr/local/lib/python3.12/dist-packages (from dask-cuda) (3.0.0)
Requirement already satisfied: dask==2026.1.1 in /usr/local/lib/python3.12/dist-packages (from rapids-dask-dependency==26.2.*->dask-)
Requirement already satisfied: distributed==2026.1.1 in /usr/local/lib/python3.12/dist-packages (from rapids-dask-dependency==26.2.*->dask-)
Requirement already satisfied: cloudpickle>=3.0.0 in /usr/local/lib/python3.12/dist-packages (from dask==2026.1.1->rapids-dask-dependency)
Requirement already satisfied: fsspec>=2021.09.0 in /usr/local/lib/python3.12/dist-packages (from dask==2026.1.1->rapids-dask-dependency)
Requirement already satisfied: packaging>=20.0 in /usr/local/lib/python3.12/dist-packages (from dask==2026.1.1->rapids-dask-dependency)
Requirement already satisfied: partd>=1.4.0 in /usr/local/lib/python3.12/dist-packages (from dask==2026.1.1->rapids-dask-dependency)
Requirement already satisfied: pyyaml>=5.3.1 in /usr/local/lib/python3.12/dist-packages (from dask==2026.1.1->rapids-dask-dependency)
Requirement already satisfied: toolz>=0.12.0 in /usr/local/lib/python3.12/dist-packages (from dask==2026.1.1->rapids-dask-dependency)
Requirement already satisfied: Jinja2>=2.10.3 in /usr/local/lib/python3.12/dist-packages (from distributed==2026.1.1->rapids-dask-dependency)
Requirement already satisfied: Locket>=1.0.0 in /usr/local/lib/python3.12/dist-packages (from distributed==2026.1.1->rapids-dask-dependency)
Requirement already satisfied: msgpack>=1.0.2 in /usr/local/lib/python3.12/dist-packages (from distributed==2026.1.1->rapids-dask-dependency)
Requirement already satisfied: psutil>=5.8.0 in /usr/local/lib/python3.12/dist-packages (from distributed==2026.1.1->rapids-dask-dependency)
Requirement already satisfied: sortedcontainers>=2.0.5 in /usr/local/lib/python3.12/dist-packages (from distributed==2026.1.1->rapids-dask-dependency)
Requirement already satisfied: tblib>=3.2.0,!<3.2.1,>=1.6.0 in /usr/local/lib/python3.12/dist-packages (from distributed==2026.1.1->rapids-dask-dependency)
Requirement already satisfied: tornado>=6.2.0 in /usr/local/lib/python3.12/dist-packages (from distributed==2026.1.1->rapids-dask-dependency)
Requirement already satisfied: urllib3>=1.26.5 in /usr/local/lib/python3.12/dist-packages (from distributed==2026.1.1->rapids-dask-dependency)
Requirement already satisfied: numba>=0.60.0 in /usr/local/lib/python3.12/dist-packages (from numba-cuda<0.23.0,>=0.22.1->dask-cuda)
Requirement already satisfied: cuda-bindings<14.0.0,>=12.9.1 in /usr/local/lib/python3.12/dist-packages (from numba-cuda<0.23.0,>=0.22.1->dask-cuda)
Requirement already satisfied: python-dateutil>=2.8.2 in /usr/local/lib/python3.12/dist-packages (from pandas>=1.3->dask-cuda) (2.9.0)
Requirement already satisfied: pytz>=2020.1 in /usr/local/lib/python3.12/dist-packages (from pandas>=1.3->dask-cuda) (2025.2)
Requirement already satisfied: tzdata>=2022.7 in /usr/local/lib/python3.12/dist-packages (from pandas>=1.3->dask-cuda) (2026.2)
Requirement already satisfied: cuda-pathfinder~=1.1 in /usr/local/lib/python3.12/dist-packages (from cuda-bindings<14.0.0,>=12.9.1->dask-cuda)
Requirement already satisfied: llvmlite<0.44,>=0.43.0dev0 in /usr/local/lib/python3.12/dist-packages (from numba>=0.60.0->numba-cuda)
Requirement already satisfied: six>=1.5 in /usr/local/lib/python3.12/dist-packages (from python-dateutil>=2.8.2->pandas>=1.3->dask-cuda)
Requirement already satisfied: MarkupSafe>=2.0 in /usr/local/lib/python3.12/dist-packages (from Jinja2>=2.10.3->distributed==2026.1.1)
```

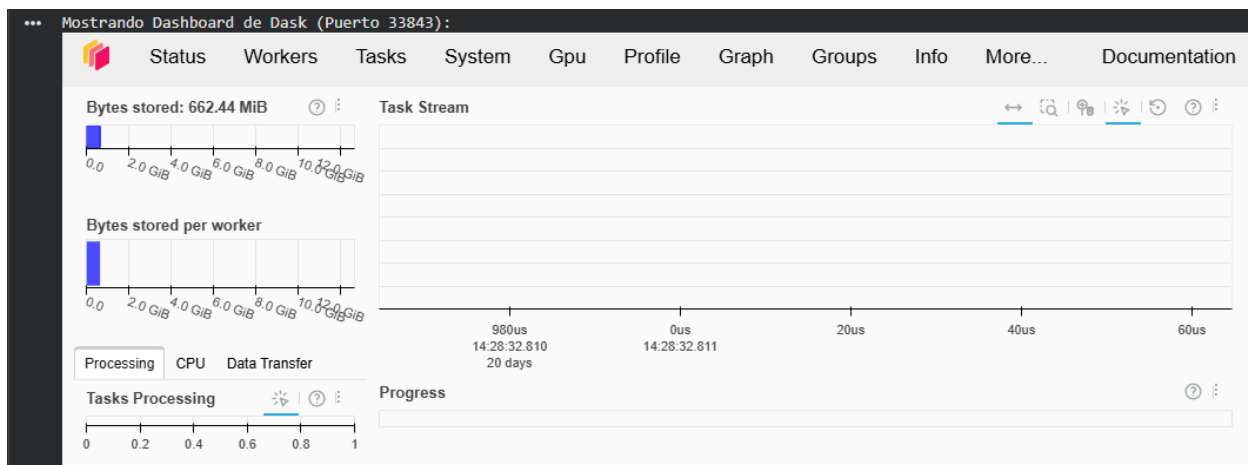
```

from dask_cuda import LocalCUDACluster
from dask.distributed import Client
# Inicializa el clúster usando todas las GPUs disponibles
cluster = LocalCUDACluster()
client = Client(cluster)
# Ver información del clúster (puertos, memoria, etc.)
print(client)

/usr/local/lib/python3.12/dist-packages/distributed/node.py:188: UserWarning: Port 8787 is already in use.
Perhaps you already have a cluster running?
Hosting the HTTP server on port 35787 instead
  warnings.warn(
INFO:distributed.scheduler:State start
INFO:distributed.scheduler: Scheduler at: tcp://127.0.0.1:38323
INFO:distributed.scheduler: dashboard at: http://127.0.0.1:35787/status
INFO:distributed.scheduler:Registering Worker plugin shuffle
INFO:distributed.nanny: Start Nanny at: 'tcp://127.0.0.1:43673'
INFO:distributed.scheduler:Register worker addr: tcp://127.0.0.1:40139 name: 0
INFO:distributed.scheduler:Starting worker compute stream, tcp://127.0.0.1:40139
INFO:distributed.core:Starting established connection to tcp://127.0.0.1:38400
INFO:distributed.scheduler:Receive client connection: Client-75c83671-5bd2-11f1-86a4-0242ac1c000c
INFO:distributed.core:Starting established connection to tcp://127.0.0.1:38410
<Client: 'tcp://127.0.0.1:38323' processes=1 threads=1, memory=12.67 GiB>

```

Visualización del Dashboard



```

from dask_cuda import LocalCUDACluster
from dask.distributed import Client
import cupy as cp # Librería para arreglos en GPU compatible con Dask
# 1. Crear un clúster donde dask-cuda detecta automáticamente las GPUs disponibles
cluster = LocalCUDACluster()

# 2. Conectar un cliente de Dask al clúster de GPUs
client = Client(cluster)
# 3. "Hola Mundo" funcional: Crear un arreglo en cada GPU y sumarlo
def hola_gpu():
    # cp.cuda.Device().id permite saber qué GPU está respondiendo [15]
    return f"Hola desde la GPU {cp.cuda.Device().id}"
# Ejecutar la función en todos los trabajadores (GPUs)
resultados = client.run(hola_gpu)
print(resultados)

/usr/local/lib/python3.12/dist-packages/distributed/node.py:188: UserWarning: Port 8787 is already in use.
Perhaps you already have a cluster running?
Hosting the HTTP server on port 45091 instead
  warnings.warn(
INFO:distributed.scheduler:State start
INFO:distributed.scheduler: Scheduler at: tcp://127.0.0.1:33325
INFO:distributed.scheduler: dashboard at: http://127.0.0.1:45091/status
INFO:distributed.scheduler:Registering Worker plugin shuffle
INFO:distributed.nanny: Start Nanny at: 'tcp://127.0.0.1:38365'
INFO:distributed.scheduler:Register worker addr: tcp://127.0.0.1:41673 name: 0
INFO:distributed.scheduler:Starting worker compute stream, tcp://127.0.0.1:41673
INFO:distributed.core:Starting established connection to tcp://127.0.0.1:51520
INFO:distributed.scheduler:Receive client connection: Client-7920a399-5bd2-11f1-86a4-0242ac1c000c
INFO:distributed.core:Starting established connection to tcp://127.0.0.1:51528
{'tcp://127.0.0.1:41673': 'Hola desde la GPU 0'}

```

Ejercicio 1: Particionamiento de Datos (Fase 'P' de PCAM)

```
Validando salidas con diferentes tamaños de chunks y computando en paralelo:
Procesando con chunks de tamaño: 100
Estructura de chunks: ((100, 100, 100, 100, 100, 100, 100, 100, 100, 100, 100, 100, 100, 100, 100, 100, 100, 100, 100, 100), (100, 100, 100, 100, 100, 100, 100, 100, 100, 100, 100, 100, 100, 100, 100, 100, 100, 100, 100, 100))
Suma calculada: 50000394.89, Tiempo de ejecución: 88.0253 segundos

Procesando con chunks de tamaño: 200
Estructura de chunks: ((200, 200, 200, 200, 200, 200, 200, 200, 200, 200, 200, 200, 200, 200, 200, 200, 200, 200, 200, 200), (200, 200, 200, 200, 200, 200, 200, 200, 200, 200, 200, 200, 200, 200, 200, 200, 200, 200, 200, 200))
Suma calculada: 50000394.89, Tiempo de ejecución: 22.7070 segundos

Procesando con chunks de tamaño: 300
Estructura de chunks: ((300, 300, 300, 300, 300, 300, 300, 300, 300, 300, 300, 300, 300, 300, 300, 300, 300, 300, 300, 300), (300, 300, 300, 300, 300, 300, 300, 300, 300, 300, 300, 300, 300, 300, 300, 300, 300, 300, 300, 300))
Suma calculada: 50000394.89, Tiempo de ejecución: 11.7406 segundos

Procesando con chunks de tamaño: 400
Estructura de chunks: ((400, 400, 400, 400, 400, 400, 400, 400, 400, 400, 400, 400, 400, 400, 400, 400, 400, 400, 400, 400), (400, 400, 400, 400, 400, 400, 400, 400, 400, 400, 400, 400, 400, 400, 400, 400, 400, 400, 400, 400))
Suma calculada: 50000394.89, Tiempo de ejecución: 7.3803 segundos

Procesando con chunks de tamaño: 500
Estructura de chunks: ((500, 500, 500, 500, 500, 500, 500, 500, 500, 500, 500, 500, 500, 500, 500, 500, 500, 500, 500, 500), (500, 500, 500, 500, 500, 500, 500, 500, 500, 500, 500, 500, 500, 500, 500, 500, 500, 500, 500, 500))
Suma calculada: 50000394.89, Tiempo de ejecución: 5.4640 segundos

Procesando con chunks de tamaño: 600
Estructura de chunks: ((600, 600, 600, 600, 600, 600, 600, 600, 600, 600, 600, 600, 600, 600, 600, 600, 600, 600, 600, 600), (600, 600, 600, 600, 600, 600, 600, 600, 600, 600, 600, 600, 600, 600, 600, 600, 600, 600, 600, 600))
Suma calculada: 50000394.89, Tiempo de ejecución: 4.0837 segundos

Procesando con chunks de tamaño: 700
Estructura de chunks: ((700, 700, 700, 700, 700, 700, 700, 700, 700, 700, 700, 700, 700, 700, 700, 700, 700, 700, 700, 700), (700, 700, 700, 700, 700, 700, 700, 700, 700, 700, 700, 700, 700, 700, 700, 700, 700, 700, 700, 700))
Suma calculada: 50000394.89, Tiempo de ejecución: 3.9160 segundos

Procesando con chunks de tamaño: 800
Estructura de chunks: ((800, 800, 800, 800, 800, 800, 800, 800, 800, 800, 800, 800, 800, 800, 800, 800, 800, 800, 800, 800), (800, 800, 800, 800, 800, 800, 800, 800, 800, 800, 800, 800, 800, 800, 800, 800, 800, 800, 800, 800))
Suma calculada: 50000394.89, Tiempo de ejecución: 3.2942 segundos

Procesando con chunks de tamaño: 900
Estructura de chunks: ((900, 900, 900, 900, 900, 900, 900, 900, 900, 900, 900, 900, 900, 900, 900, 900, 900, 900, 900, 900), (900, 900, 900, 900, 900, 900, 900, 900, 900, 900, 900, 900, 900, 900, 900, 900, 900, 900, 900, 900))
Suma calculada: 50000394.89, Tiempo de ejecución: 2.6756 segundos

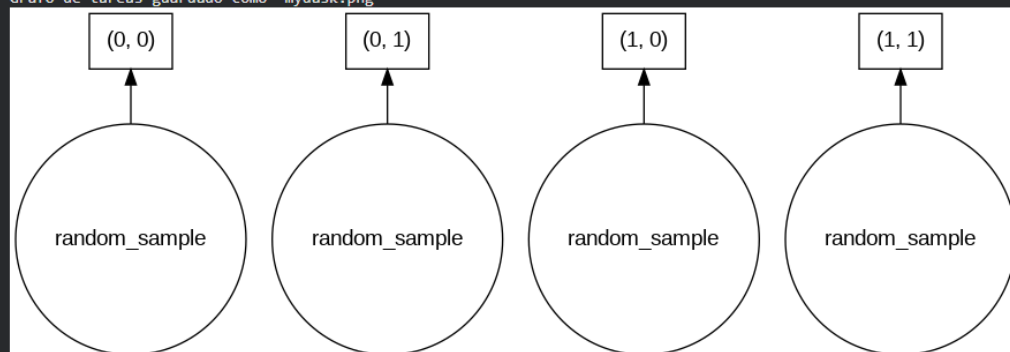
Procesando con chunks de tamaño: 1000
Estructura de chunks: ((1000, 1000, 1000, 1000, 1000, 1000, 1000, 1000, 1000, 1000, 1000, 1000, 1000, 1000, 1000, 1000, 1000, 1000, 1000, 1000), (1000, 1000, 1000, 1000, 1000, 1000, 1000, 1000, 1000, 1000, 1000, 1000, 1000, 1000, 1000, 1000, 1000, 1000, 1000, 1000))
Suma calculada: 50000394.89, Tiempo de ejecución: 2.0914 segundos
```

```

Resultados por tamaño de chunk:
  Chunk Size 100: Sum=50000394.89, Time=88.0253s
  Chunk Size 200: Sum=50000394.89, Time=22.7070s
  Chunk Size 300: Sum=50000394.89, Time=11.7406s
  Chunk Size 400: Sum=50000394.89, Time=7.3803s
  Chunk Size 500: Sum=50000394.89, Time=5.4640s
  Chunk Size 600: Sum=50000394.89, Time=4.0837s
  Chunk Size 700: Sum=50000394.89, Time=3.9160s
  Chunk Size 800: Sum=50000394.89, Time=3.2942s
  Chunk Size 900: Sum=50000394.89, Time=2.6756s
  Chunk Size 1000: Sum=50000394.89, Time=2.0914s

```

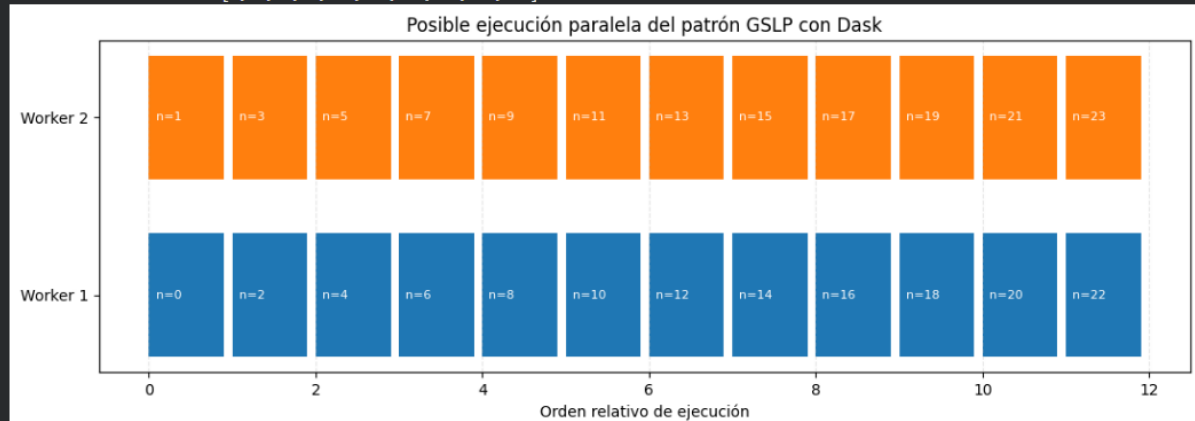
Grafo de tareas guardado como 'mydask.png'



Suma calculada de forma diferida (sin compute): `dask.array<sum-aggregate, shape=(), dtype=float64, chunksize=(), chunktype=numpy.ndarray>`

Ejercicio 2: Implementación del Patrón GSLP con Dask:

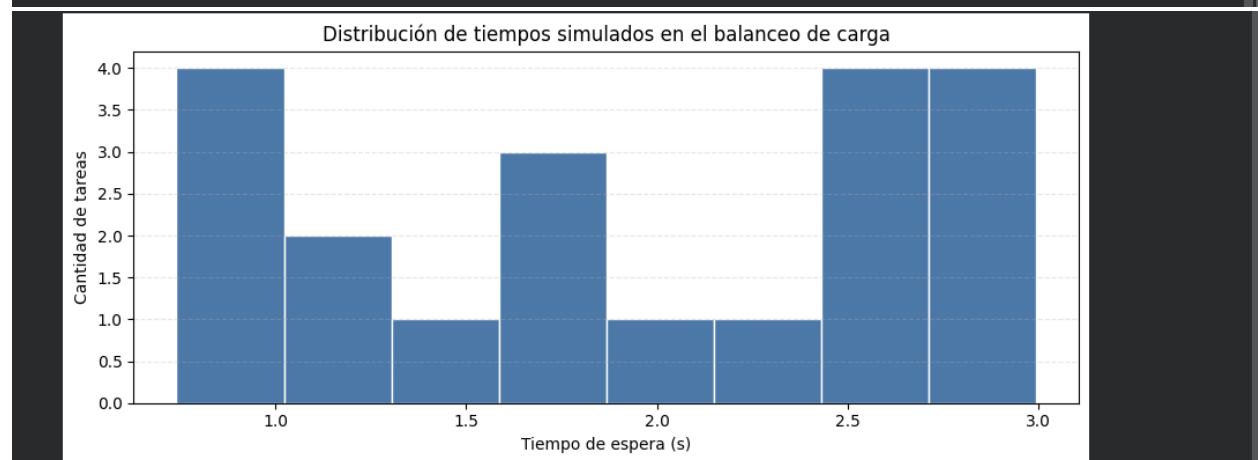
```
*** /usr/local/lib/python3.12/dist-packages/distributed/node.py:188: UserWarning: Port 8787 is already in use.  
Perhaps you already have a cluster running?  
Hosting the HTTP server on port 43825 instead  
warnings.warn(  
INFO:distributed.scheduler:State start  
INFO:distributed.scheduler: Scheduler at: tcp://127.0.0.1:41529  
INFO:distributed.scheduler: dashboard at: http://127.0.0.1:43825/status  
INFO:distributed.scheduler:Registering Worker plugin shuffle  
INFO:distributed.nanny: Start Nanny at: 'tcp://127.0.0.1:39701'  
INFO:distributed.nanny: Start Nanny at: 'tcp://127.0.0.1:33943'  
INFO:distributed.scheduler:Register worker addr: tcp://127.0.0.1:44421 name: 0  
INFO:distributed.scheduler:Starting worker compute stream, tcp://127.0.0.1:44421  
INFO:distributed.core:Starting established connection to tcp://127.0.0.1:45292  
INFO:distributed.scheduler:Register worker addr: tcp://127.0.0.1:37073 name: 1  
INFO:distributed.scheduler:Starting worker compute stream, tcp://127.0.0.1:37073  
INFO:distributed.core:Starting established connection to tcp://127.0.0.1:45300  
INFO:distributed.scheduler:Receive client connection: Client-d45da93b-5bd2-11f1-86a4-0242ac1c000c  
INFO:distributed.core:Starting established connection to tcp://127.0.0.1:45310  
Primeros 10 resultados: [0, 1, 4, 9, 16, 25, 36, 49, 64, 81]
```



Ejercicio 3: Balanceo de Carga (Fase 'Mapping' de PCAM)

... Procesando tareas... observa el balanceo de carga en el Dashboard.

	tarea	tiempo_s	resultado
0	0	1.159843	Tarea 0 completada en 1.16s
1	1	1.043235	Tarea 1 completada en 1.04s
2	2	2.150642	Tarea 2 completada en 2.15s
3	3	1.723039	Tarea 3 completada en 1.72s
4	4	1.908899	Tarea 4 completada en 1.91s
5	5	0.911796	Tarea 5 completada en 0.91s
6	6	2.714435	Tarea 6 completada en 2.71s
7	7	1.593054	Tarea 7 completada en 1.59s
8	8	2.896156	Tarea 8 completada en 2.90s
9	9	0.740621	Tarea 9 completada en 0.74s
10	10	2.670967	Tarea 10 completada en 2.67s
11	11	2.677696	Tarea 11 completada en 2.68s
12	12	1.367256	Tarea 12 completada en 1.37s
13	13	0.858724	Tarea 13 completada en 0.86s
14	14	0.941086	Tarea 14 completada en 0.94s
15	15	1.645508	Tarea 15 completada en 1.65s
16	16	2.641121	Tarea 16 completada en 2.64s
17	17	2.766979	Tarea 17 completada en 2.77s
18	18	2.993239	Tarea 18 completada en 2.99s
19	19	2.508392	Tarea 19 completada en 2.51s



PARTE 2:

a) Los entorno de nube AWS poseen las familias AWS G7e P5,Trn2, P6, VT1 usadas para para trabajar con IA y GPUs, una vez analizadas realice una tabla con las siguientes características:

	Familia	Características principales	Uso	Cantidad de GPUs / aceleradores	Cantidad de CPU (vCPUs)	Cantidad de memoria	Almacenamiento mínimo (Local NVMe)
0	G7e*	GPU orientada a inferencia y gráficos; referencia pública más cercana: G6e con NVIDIA L40S y Nitro	Inferencia de IA, spatial computing, visualización acelerada	Hasta 8 GPUs L40S	Hasta 192	Hasta 1.536 TB	Hasta 7.6 TB
1	P5	Instancias GPU con NVIDIA H100/H200 para entrenamiento e HPC	Entrenamiento de LLMs, HPC, simulación científica	Hasta 8 GPUs H100/H200	Hasta 192	Hasta 2 TiB	8 x 3.84 TB NVMe SSD
2	Trn2	Aceleradores AWS Trainium2 para IA generativa y entrenamiento a gran escala	Entrenamiento e inferencia de modelos fundacionales	Hasta 16 chips Trainium2	Hasta 192	Hasta 1.5 TB	4 x 1.92 TB NVMe SSD
3	P6	Instancias Blackwell / Blackwell Ultra para IA de frontera	Entrenamiento e inferencia de modelos MoE y reasoning	Hasta 8 GPUs Blackwell	Hasta 192	Hasta 4 TiB	8 x 3.84 TB NVMe SSD
4	VT1	Aceleración de transcodificación de video en tiempo real	Broadcast, videoconferencia y transcoding just-in-time	Hasta 8 aceleradores Xilinx U30	Hasta 96	Hasta 192 GB	No especificado / orientado a EBS
Nota: G7e se deja como referencia equivalente pública de la familia gráfica G6e.							

b) Con base en la tabla generada seleccione las mejores combinaciones de familias específicas para ejecutar los ejercicios propuestos (1,2 y 3 - Parte 1), tomar como mínimo los siguientes encabezados:

	Ejercicio	Prioridad arquitectónica	Familia AWS recomendada	Motivo técnico
0	1. Particionamiento de datos	Capacidad de memoria y ancho de banda	P6	Es la opción más robusta para arreglos grandes y cargas de cómputo intensivo; su memoria local y escala de GPU ayudan a manejar particiones grandes sin degradar tanto el rendimiento.
1	2. GSLP con Dask	Bajo costo por tarea y buena latencia de orquestación	G7e	El ejemplo es ligero y depende más del scheduler que de cómputo crudo. Una GPU de gama intermedia permite ejecutar varios workers sin pagar la sobrecapacidad de P6 o P5.
2	3. Balanceo de carga	Elasticidad y monitoreo de workers	G7e	La carga está dominada por tareas heterogéneas y esperas variables; conviene una familia costo-eficiente para observar el reparto de trabajo sin sobredimensionar infraestructura.
P6 para el ejercicio más pesado de datos; G7e para las demos de Dask por costo/eficiencia.				

c) Ahora requerimos calcular el costo mensual y anual de estos servicios (Ejercicios 1,2 y 3) que variables son importantes respecto a los parámetros de la calculadora de AWS en infraestructura y que parámetros son importantes en modo IA (Usar la calculadora de AWS <https://calculator.aws/#/>):

	Modo	Variables importantes	Parámetros que se ajustan	Fórmula práctica												
0	Infraestructura	tipo de instancia, número de instancias, región, horas/mes, compra On-Demand/Spot/Savings Plans, EBS, snapshots, transferencias de datos	tamaño de nodo, escalado horizontal, almacenamiento, red entre zonas, reserva o pago por demanda	$\text{costo mensual} = \text{tarifa_hora} \times \text{horas_uso} \times \text{cantidad_instancias} + \text{almacenamiento} + \text{red} + \text{I/O}$												
1	IA / GPU	número de GPUs o aceleradores, memoria por GPU, VRAM total, entrenamiento vs inferencia, tiempo de cómputo por experimento, EFA / UltraCluster, checkpoints	model size, batch size, precisión (FP16/BF16/FP8), distribución del entrenamiento, almacenamiento temporal y ancho de banda	$\text{costo mensual} = (\text{tarifa_hora GPU} \times \text{horas_entrenamiento}) + \text{almacenamiento de datasets} + \text{checkpoints} + \text{red}$												
<table><tr><th>Ejercicio</th><th>Mensual</th><th>Anual</th></tr><tr><td>0 1</td><td>$\text{tarifa_hora_1} \times \text{horas_1}$</td><td>$12 \times \text{mensual}$</td></tr><tr><td>1 2</td><td>$\text{tarifa_hora_2} \times \text{horas_2}$</td><td>$12 \times \text{mensual}$</td></tr><tr><td>2 3</td><td>$\text{tarifa_hora_3} \times \text{horas_3}$</td><td>$12 \times \text{mensual}$</td></tr></table>					Ejercicio	Mensual	Anual	0 1	$\text{tarifa_hora_1} \times \text{horas_1}$	$12 \times \text{mensual}$	1 2	$\text{tarifa_hora_2} \times \text{horas_2}$	$12 \times \text{mensual}$	2 3	$\text{tarifa_hora_3} \times \text{horas_3}$	$12 \times \text{mensual}$
Ejercicio	Mensual	Anual														
0 1	$\text{tarifa_hora_1} \times \text{horas_1}$	$12 \times \text{mensual}$														
1 2	$\text{tarifa_hora_2} \times \text{horas_2}$	$12 \times \text{mensual}$														
2 3	$\text{tarifa_hora_3} \times \text{horas_3}$	$12 \times \text{mensual}$														
Use AWS Calculator para reemplazar los parámetros por valores reales.																

d) Realice una tabla con el detalle de memoria RAM requerida por cada una de las familia seleccionadas en el punto anterior y analice las ventajas y desventajas de la selección realizada:

***	Familia	RAM requerida	Ventajas		Desventajas
0	G7e*	Hasta 1.536 TB	Equilibrio entre costo y memoria para inferencia y visualización; suficiente para demos de Dask y particionado moderado.	No está pensada para los modelos más grandes ni para entrenamiento extremo de frontera.	
1	P6	Hasta 4 TiB	Máxima holgura para datos masivos, entrenamiento distribuido y cargas de IA de frontera.	Costo muy alto; para ejercicios sencillos puede sobredimensionar la solución.	
2	VT1	Hasta 192 GB	Muy eficiente para su caso de uso específico: transcodificación de video en tiempo real.	No es la mejor opción para ejercicios de IA general o entrenamiento GPU tradicional.	
3	Tm2	Hasta 1.5 TB	Excelente relación precio/rendimiento para entrenamiento e inferencia con Trainium2 y gran memoria total.	Requiere adaptación al ecosistema Neuron; no es una GPU CUDA clásica.	
Conclusión: para la práctica, P6 domina el ejercicio de datos grandes, mientras que G7e cubre de forma más eficiente las demostraciones de Dask.					